

Нейронная сеть Хопфилда

Последнее обновление: 17 февраля 2026 года

Сеть Хопфилда — это разновидность рекуррентной нейронной сети, предназначенная для хранения и воспроизведения паттернов путем имитации ассоциативной памяти человеческого мозга. Она работает за счет многократного обновления взаимосвязанных нейронов до тех пор, пока система не перейдет в стабильное состояние, соответствующее сохраненному паттерну. Это делает ее особенно полезной для восстановления полной информации из неполных или зашумленных входных данных.

- Однослойная сеть, в которой каждый нейрон связан со всеми остальными нейронами
- Входные и выходные данные имеют одинаковый размер (автоассоциативная модель)
- Использует симметричные веса без внутренних связей
- Приходит к стабильному состоянию в процессе минимизации энергии

Типы сетей Хопфилда

Сети Хопфилда классифицируются в зависимости от того, как представлены выходные данные нейронов и как меняется динамика сети с течением времени.

1. Дискретная сеть Хопфилда

Дискретная сеть Хопфилда — это полностью связанная рекуррентная нейронная сеть, в которой каждый нейрон связан со всеми остальными нейронами (кроме самого себя). Сеть обновляет состояния нейронов дискретными шагами и выдает конечное число различных выходных данных.

- Функционирует как система автоассоциативной памяти, которая итеративно обновляет нейроны (синхронно или асинхронно).
- Приближается к стабильному равновесному состоянию за счет минимизации энергии.
- Имеет ограниченную емкость памяти ($0,15n$ паттернов) и при перегрузке может генерировать ложные состояния.

Дискретные сети могут быть представлены в двух возможных вариантах:

- **Двоичное представление:** Выходные значения равны 0 или 1
- **Биполярное представление:** Выходные значения равны -1 или +1

Матрица весов в дискретной сети Хопфилда подчиняется двум важным правилам:

$$w_{ij} = w_{ji} \text{ (Симметричные веса)}$$

$$w_{ii} = 0 \text{ (Без самоподключений)}$$

Эти свойства обеспечивают существование четко определенной функции энергии, которая гарантирует сходимость к стабильному состоянию.

2. Непрерывная сеть Хопфилда

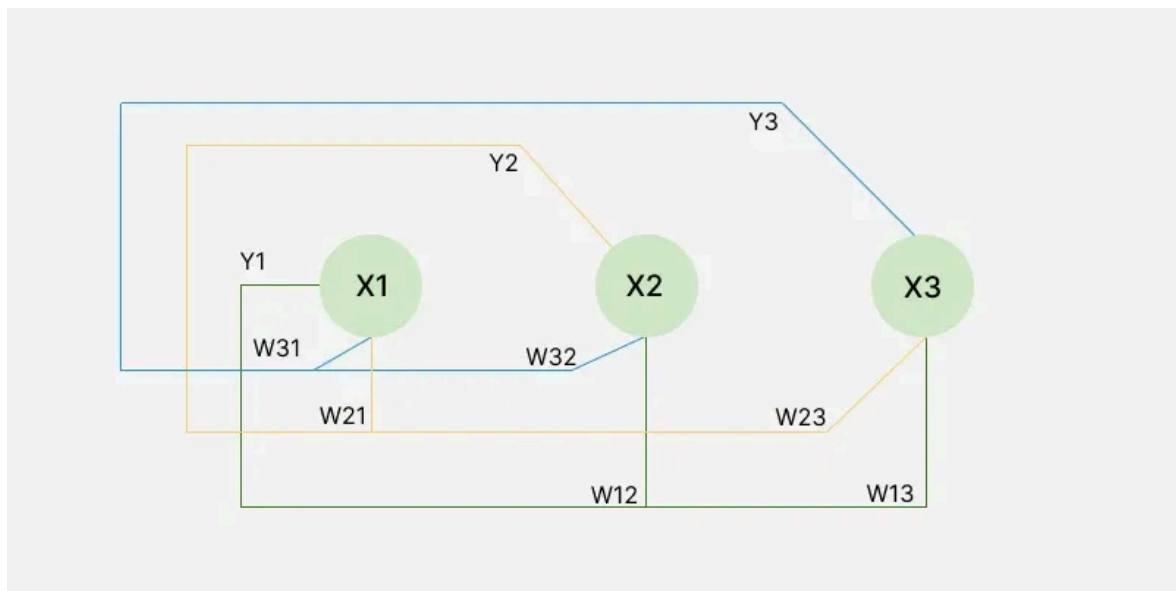
Непрерывная сеть Хопфилда отличается от дискретной тем, что выходные значения нейронов не ограничены фиксированными значениями. Вместо этого они плавно изменяются в диапазоне от 0 до 1.

- Работает в режиме непрерывного времени и управляется дифференциальными уравнениями.
- Функция энергии плавно уменьшается с течением времени, обеспечивая стабильную сходимость.
- Хорошо подходит для решения задач оптимизации с ограничениями, динамика которых более плавная, чем в дискретных сетях.

Архитектура сети Хопфилда

Сеть Хопфилда — это однослойная полносвязная рекуррентная нейронная сеть, в которой нейроны обновляются посредством обратной связи до тех пор, пока не будет достигнуто стабильное состояние. Ее симметричная структура обеспечивает ассоциативную память и конвергенцию на основе энергии. Основные структурные особенности сети Хопфилда:

- **Полностью связная сеть:** Каждый нейрон связан со всеми остальными нейронами в сети.
- **Отсутствие самосвязей:** Нейрон не связан сам с собой.
- **Симметричные веса:** Матрица весов симметрична.
- **Сохранение состояния:** Каждый нейрон сохраняет свое текущее состояние до тех пор, пока его не выберут для обновления.
- **Асинхронное обновление:** Нейроны выбираются и обновляются по одному, что повышает стабильность и обеспечивает сходимость.



Discrete Hopfield Network Architecture

For a network with n neurons:

- $X = [x_1, x_2, \dots, x_n]$: Input to the n given neurons
- $Y = [y_1, y_2, \dots, y_n]$: Output obtained from the n given neurons
- W_{ij} : denotes the connection strength between neuron i and neuron j

How Hopfield Networks Learn and Recall Patterns

Training a Hopfield Network means storing patterns in the weight matrix using the Hebbian learning rule. After training, the network can recall stored patterns from partial or noisy inputs through an iterative testing process.

Training Algorithm

The Hopfield Network stores patterns using the Hebbian learning rule, which strengthens connections between neurons that activate together.

For storing a set of P input patterns the weight matrix is computed using the Hebbian outer-product rule.

$$S^{(p)} = \left[s_1^{(p)}, s_2^{(p)}, \dots, s_n^{(p)} \right], \quad p = 1, 2, \dots, P$$

Here $s_i^{(p)}$ is the state of the i -th neuron in that p -th pattern

Case 1: Binary Patterns (0/1)

For binary patterns, values are first converted into bipolar form before applying the Hebbian rule.

$$w_{ij} = \sum_{p=1}^P \left(2s_i^{(p)} - 1 \right) \left(2s_j^{(p)} - 1 \right), \quad i \neq j$$

Here

- $(2s_i^{(p)} - 1)$ converts binary values (0,1) into bipolar values (-1,+1).
- Self-connections are removed by setting w_{ii}

Case 2: Bipolar Patterns (-1/+1)

For bipolar patterns, weights are directly computed using the outer-product learning rule.

$$w_{ij} = \sum_{p=1}^P s_i^{(p)} s_j^{(p)}, \quad i \neq j$$

Here $s_i^{(p)} \in \{-1, +1\}$ and the weights are symmetric ($w_{ij} = w_{ji}$)

Testing (Recall) Algorithm

After training, the Hopfield Network recalls stored patterns by iteratively updating neuron states until convergence to a stable state (energy minimum). Asynchronous updating is typically used to guarantee convergence.

Step 1: Initialize the Network

Set the initial state equal to the input vector

Step 2: Compute Net Input

For the selected neuron i , compute the net input:

$$y_{in_i} = \sum_{j=1}^n w_{ij} y_j$$

Step 3: Apply Activation Function

For binary representation (0/1):

$$y_i = \begin{cases} 1 & \text{if } y_{in_i} > \theta_i \\ y_i & \text{if } y_{in_i} = \theta_i \\ 0 & \text{if } y_{in_i} < \theta_i \end{cases}$$

For bipolar representation (-1/+1):

$$y_i = \begin{cases} +1 & \text{if } y_{in_i} \geq 0 \\ -1 & \text{if } y_{in_i} < 0 \end{cases}$$

Step 4: Update and Feedback

In asynchronous updating, one neuron is updated at a time by replacing its old state with the new value and the updated output is then fed back to all other neurons in the network.

Step 5: Check for Convergence

The network is said to have converged when no neuron changes its state during a complete update cycle, indicating that a stable state (local energy minimum) has been reached.

Energy Function in Hopfield Network

Energy function (Lyapunov function) assigns an energy value to every possible state of the network. During the updating process, the energy either decreases or remains constant, ensuring that the network eventually converges to a stable state (local minimum).

1. Energy Function in Discrete Hopfield Network

For a network with symmetric weights and no self-connections, the energy function is defined as:

$$E = -\frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n w_{ij} x_i x_j + \sum_{i=1}^n \theta_i x_i$$

where

- w_{ij} : weight between neuron i and neuron j
- x_i, x_j : states of neurons
- θ_i : threshold of neuron i

In asynchronous updating, the energy never increases and the network converges to a stable memory state at a local minimum.

2. Energy Function in Continuous Hopfield Network

In the continuous network neuron outputs vary smoothly and the energy function is defined as:

$$E = -\frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n w_{ij} v_i v_j + \sum_{i=1}^n \theta_i v_i$$

where v_i represent continuous output of neuron i.

Convergence Condition

For continuous dynamics, convergence is guaranteed if:

$$\frac{dE}{dt} \leq 0$$

This means the energy decreases over time. The neuron dynamics are governed by:

$$\frac{du_i}{dt} = -\frac{u_i}{\tau} + \sum_{j=1}^n w_{ij}v_j + \theta_i$$

where

- u_i : internal state
- τ : time constant
- $v_i = g(u_i)$: activation function

Step By Step Implementation

Here we implement Hopfield Neural Network

Step 1: Import Required Libraries

Here we import all the necessary libraries for building and running the Hopfield Network

```
!pip install scikit-image tqdm

import numpy as np
from matplotlib import pyplot as plt
from skimage.filters import threshold_mean
from tqdm import tqdm
from tensorflow.keras.datasets import mnist
```

Step 2: Define Hopfield Network and Train Weights

Here we define the Hopfield Network class and implement the weight training method.

- We calculate the weight matrix using Hebbian learning with mean-centering.
- The diagonal is set to 0 to avoid self-connections.

```
class HopfieldNetwork:
    def train_weights(self, train_data):
        self.num_neuron = train_data[0].size # number of neurons
        rho = np.mean([np.sum(d) for d in train_data]) # mean activity
        W = sum(np.outer(d - rho, d - rho) for d in train_data) # Hebbian sum
        np.fill_diagonal(W, 0) # no self-connection
        self.W = W / len(train_data) # normalize by number of patterns
```

Step 3: Predict Patterns and Compute Energy

Here we implements pattern recall:

- The network updates neurons asynchronously or synchronously.
- Updates stop when the energy converges or max iterations are reached.
- Energy of the state is calculated to track convergence.

```
def predict(self, data, num_iter=20, threshold=0, asyn=True):
    pred = []
    energies = []
    for s in tqdm(data):
        s = s.copy()
        e_list = [self.energy(s)]
        for _ in range(num_iter):
            if asyn: # asynchronous update
                for _ in range(self.num_neuron):
                    i = np.random.randint(self.num_neuron)
                    s[i] = np.sign(self.W[i] @ s - threshold)
                    if s[i] == 0: s[i] = 1
            else: # synchronous update
```

```

        s = np.sign(self.W @ s - threshold)
        s[s == 0] = 1
        e_new = self.energy(s)
        e_list.append(e_new)
        if e_new == e_list[-2]: # stop if energy converged
            break
        pred.append(s)
        energies.append(e_list)
    return pred, energies

def energy(self, s):
    return -0.5 * s @ self.W @ s

```

Step 4: Preprocess Images

Convert grayscale images into bipolar format -1,+1 using a threshold.

```

def preprocess(img):
    return 2*(img > threshold_mean(img)).astype(int)-1

def reshape_img(v):
    dim = int(np.sqrt(v.size))
    return v.reshape((dim, dim))

```

Step 5: Plot Training, Input and Predicted Images

Visual comparison between stored memory, input and network output.

```

def plot_results(train, test, predicted):
    train = [reshape_img(d) for d in train]
    test = [reshape_img(d) for d in test]
    predicted = [reshape_img(d) for d in predicted]

    fig, axes = plt.subplots(len(train), 3, figsize=(8, 3*len(train)))
    if len(train) == 1: axes = [axes]
    for i in range(len(train)):
        for j, data in enumerate([train[i], test[i], predicted[i]]):
            axes[i][j].imshow(data, cmap='gray')
            axes[i][j].axis('off')
            axes[i][j].set_title(["Train", "Input", "Predicted"][j])
    plt.tight_layout()
    plt.show()

```

Step 6: Hamming Distance

Calculate the number of differing bits between predicted patterns and stored memories.

```

def hamming_distance(a, b):
    return np.sum(a != b)

```

Step 7: Load MNIST Data and Select Training/Testing Images

Pick one image per class for training and one for testing. Preprocess them.

```

(x_train, y_train), _ = mnist.load_data()

train_imgs = [preprocess(x_train[y_train==i][0]) for i in range(3)]
test_imgs = [preprocess(x_train[y_train==i][1]) for i in range(3)]

flattened_train_imgs = [img.flatten() for img in train_imgs]
flattened_test_imgs = [img.flatten() for img in test_imgs]

```

Step 8: Train the Hopfield Network

Train the weight matrix using the flattened training images.

```
model = HopfieldNetwork()
model.train_weights(flattened_train_imgs)
```

Step 9: Predict and Track Energy

Run the network on test images with asynchronous updates and record energy convergence.

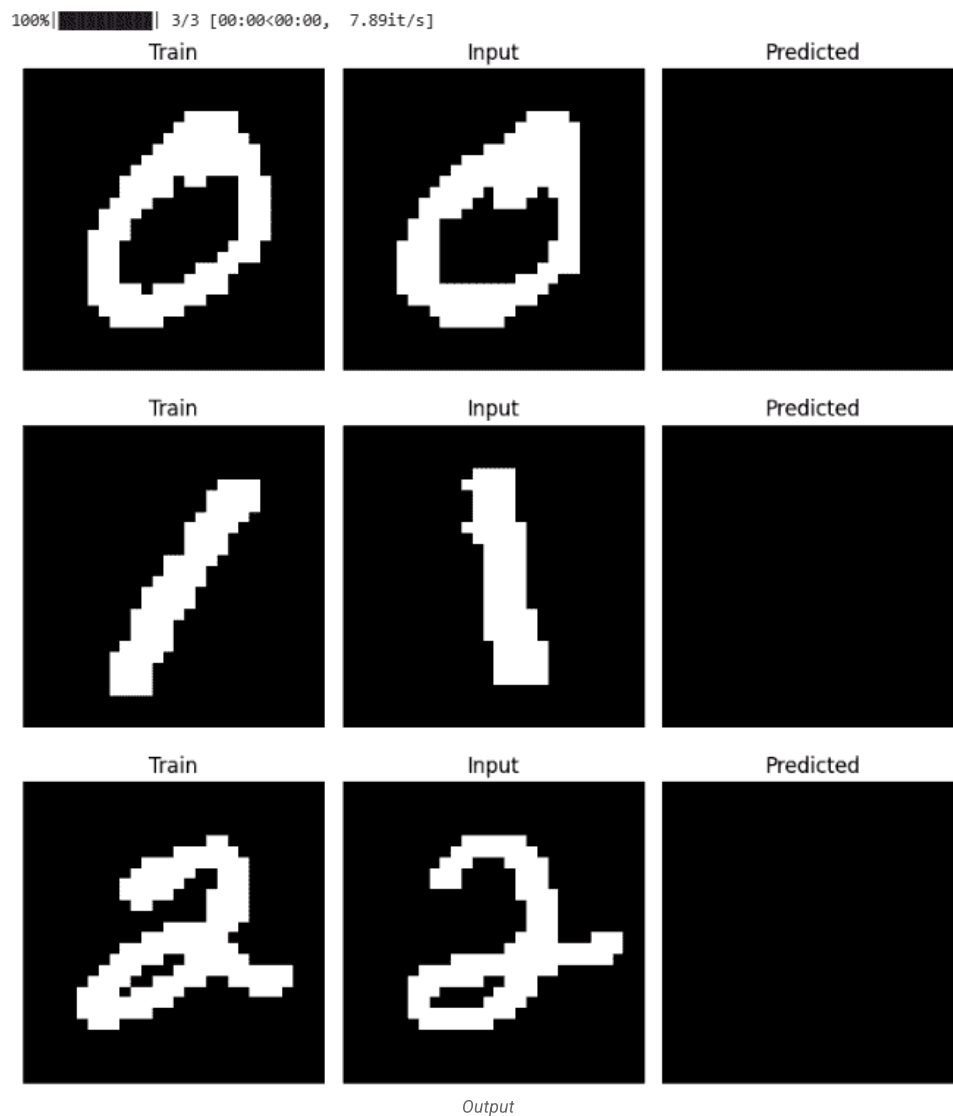
```
predicted, energies = model.predict(flattened_test_imgs, num_iter=20, asyn=True)
```

Step 10: Visualize Results

Compare training images, inputs and outputs.

```
plot_results(train_imgs, test_imgs, predicted)
```

Output:



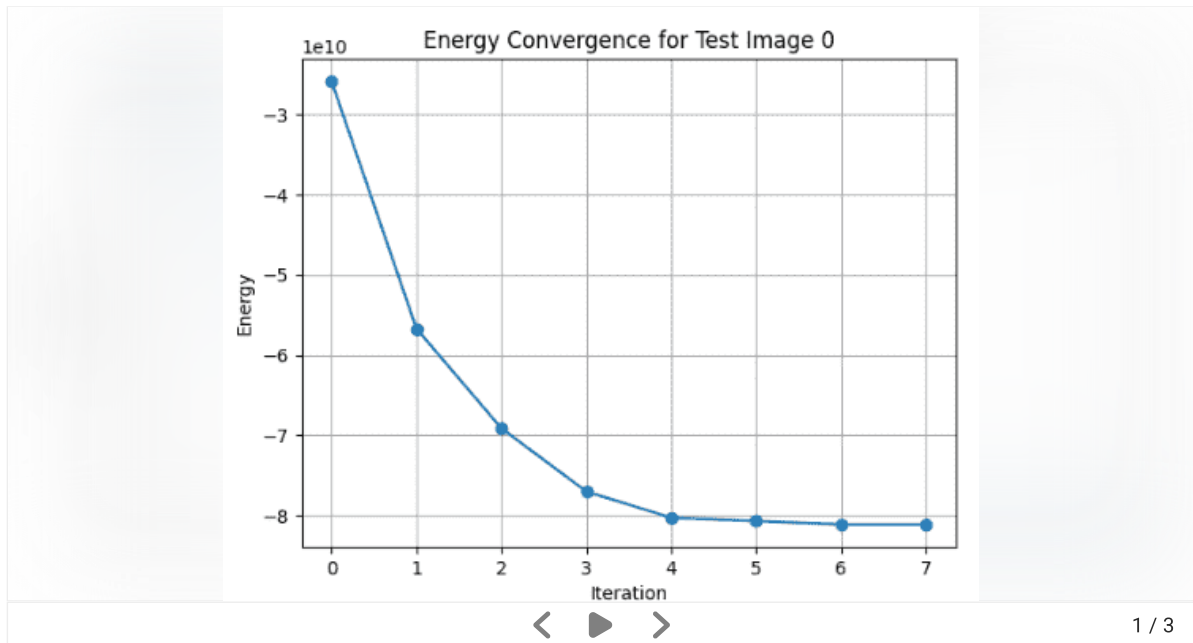
Step 11: Plot Energy Convergence

Show how energy decreases during iterative updates.

```
for i, e_list in enumerate(energies):
    plt.plot(e_list, marker='o')
```

```
plt.title(f'Energy Convergence for Test Image {i}')
plt.xlabel('Iteration')
plt.ylabel('Energy')
plt.grid(True)
plt.show()
```

Output:



These plots show how the energy of each test image changes over iterations in the Hopfield Network.

- The y-axis shows the energy value and the x-axis shows the iteration number.
- A decreasing or stable energy curve indicates the network is converging to a stable memory state

You can download full code from [here](#)

P prakha...

12

Article Tags: [Machine Learning](#) [AI-ML-DS](#) [Neural Network](#)

Explore

Machine Learning Basics

Python for Machine Learning

Feature Engineering

Supervised Learning

Unsupervised Learning

Model Evaluation and Tuning

Advanced Techniques

Machine Learning Practice

 **Corporate & Communications Address:**
A-143, 7th Floor, Sovereign Corporate
Tower, Sector- 136, Noida, Uttar
Pradesh (201305)

 **Registered Address:**
K 061, Tower K, Gulshan Vivante
Apartment, Sector 137, Noida, Gautam
Buddh Nagar, Uttar Pradesh, 201305



Company

About Us
Legal
Privacy
Policy
Contact Us
Advertise
with us
GFG
Corporate
Solution
Training
Program

Explore

POTD
Job-A-
Thon
Blogs
Nation
Skill Up

Tutorials

Programming
Languages
DSA
Web
Technology
AI, ML & Data
Science
DevOps
CS Core
Subjects
Interview
Preparation
Software and
Tools

Courses

ML and Data
Science
DSA and
Placements
Web
Development
Programming
Languages
DevOps &
Cloud
GATE
Trending
Technologies

Videos

DSA
Python
Java
C++
Web
Development
Data Science
CS Subjects

Preparation

Corner
Interview
Corner
Aptitude
Puzzles
GfG 160
System
Design